Building FemboyEngine and Neon Triangle

Everything below produces the same game from the same tree. The engine is a static library, so a build only ships the game executable, its assets and (when SFML is linked dynamically) the SFML runtime.

Platform	Compiler	Status	How
Windows 10/11 x64	MSVC 19.4x (VS 2022)	Verified: game, tests, screenshots, package	build.bat or the CMake commands below
Windows 10/11 x64	MinGW-w64 GCC 14.2 (UCRT, posix)	Toolchain installed; build script present	cmake with the MinGW SFML tree
Linux x64	GCC / Clang	CI job (build.yml), build-linux.sh	system deps + fetched SFML 3
macOS 12+	AppleClang	CI job (build.yml), build-macos.sh	Xcode CLT + fetched SFML 3
Android 7+	NDK r26+	Not supported yet — see below	—

Windows — MSVC (verified)

Requirements: Visual Studio 2022 with the "Desktop development with C++" workload, CMake 3.20+, Ninja (bundled with VS).

```
call "C:\Program Files\Microsoft Visual
Studio\2022\Community\VC\Auxiliary\Build\vcvars64.bat"
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=RelWithDebInfo ^
      -DFEMBOY_BUILD_NEONTRIANGLE=ON -DFEMBOY_BUILD_TESTS=ON
cmake --build build
build\femboy_tests.exe
build\NeonTriangle.exe
```

The SFML tree in libs/SFML-3.1.0 is picked automatically for MSVC. The copy_sfml_dlls target copies the runtime DLLs next to the executable; run it when you build individual targets with ninja -C build NeonTriangle instead of cmake --build build.

Windows — MinGW-w64

The second pre-built tree, libs/SFML-3.1.0-windows-gcc-14.2.0-mingw-64-bit/SFML-3.1.0, matches GCC 14.2 with the UCRT and posix threads. Point CMake at that compiler and Ninja:

```
cmake -S . -B build-mingw -G Ninja ^
      -DCMAKE_BUILD_TYPE=RelWithDebInfo ^
      -DCMAKE_C_COMPILER=C:/path/to/mingw64/bin/gcc.exe ^
      -DCMAKE_CXX_COMPILER=C:/path/to/mingw64/bin/g++.exe ^
      -DFEMBOY_BUILD_NEONTRIANGLE=ON
cmake --build build-mingw
```

FEMBOY_SFML_ROOT selects the tree by compiler; override it only when pointing

at a different SFML build.

First-run setup

The editor opens**Setup & Diagnostics**automatically when a required tool is

missing. Press F2 or use Help -> Setup & Diagnostics at any time.

Command-line equivalents:

OS	Check	Install
Windows	tools\setup_windows.bat	tools\setup_windows.bat -Install
Linux	./tools/setup_linux.sh	./tools/setup_linux.sh --install
macOS	./tools/setup_macos.sh	./tools/setup_macos.sh --install

SFML, ImGui, EnTT, Box2D and the other libraries are vendored by the engine;

the setup scripts only install the compiler/build tools the host needs.

Console behavior

- Windows games/editor use the Windows GUI subsystem, so they do not spawn a

console. Launch with --console (or --dev-console) to attach one.

- Linux/macOS desktop launches have no special console subsystem. A terminal

launch keeps its terminal; a desktop/Finder launch has no terminal. The

in-game console (F1 or) works on every desktop platform.

- Tests are always console executables on every platform.

Linux

SFML 3 is newer than what most distributions package, so the default is to let

CMake download and build it:

```
sudo apt install build-essential cmake ninja-build \
  libx11-dev libxrandr-dev libxcursor-dev libxi-dev libudev-dev \
```

```
    libgl1-mesa-dev libfreetype-dev libasound2-dev libvorbis-dev libflac-dev
  ./build-linux.sh
```

build-linux.sh configures -DFEMBOY_FETCH_SFML=ON, builds the game and tests,

and runs the tests. Set BUILD_DIR, BUILD_TYPE or SKIP_TESTS=1 to change

that.

macOS

```
  xcode-select --install
  brew install cmake ninja
  ./build-macos.sh
```

Apple Silicon is the reference target. Universal binaries can be produced with

-DCMAKE_OSX_ARCHITECTURES="arm64;x86_64", at the cost of a longer SFML build.

FEMBOY_NATIVE_ARCH=ON enables -march=native//arch:AVX2 for release builds.

It is off by default so a binary built on one machine can run on another.

Android — status and what it would take

SFML 3 does**not**provide an Android or iOS backend: its window, input and

audio implementations assume a desktop OS. android/build_android.bat and

cmake/android_toolchain.cmake are scaffolding for the NDK, but they cannot

produce an APK until the engine has a platform layer that does not depend on

SFML's desktop window. CMake stops with a clear error for ANDROID/IOS rather

than failing deep inside SFML.

The work, in order:

1. **Platform layer.**An android_main entry point (ANativeActivity,

android_native_app_glue), an EGL surface, and a TouchInput source. The

engine's IRenderer interface is already the seam: a GLESRenderer backend

implements it the way SFMLRenderer does today.

2. **Assets.**Pack assets/ into the APK, expose it through AssetManager

with AAssetManager instead of std::ifstream paths.

3. **Audio.**SFML 3 audio is miniaudio-based; miniaudio has AAudio and OpenSL ES

backends, but the engine still needs an SFML platform layer or a direct

miniaudio playback path before sf::SoundBuffer works on Android.

4. **Gradle.**A minimal build.gradle with externalNativeBuild.cmake, a manifest and an icon. None of this exists yet.

That is a port, not a build-flag change: plan it as a milestone, not a pre-release checklist item. The honest state of the tree is "Windows playable, Linux/macOS buildable in CI, Android not started".

Packaging

```
cmake --build build --target package
```

CPack writes a ZIP into the build directory with the game executable, generated assets, the third-party notices and, on Windows, the four release SFML DLLs. The release is deliberately ZIP-only: no installer or registry writes are part of the package. The release folder ships the ZIP plus SHA256SUMS.txt.

Licence of the build output

The engine is MPL 2.0: you may ship a commercial, closed-source game, but modifications to files under engine/ must be published under the MPL. Files under Projects/ are proprietary and are not affected. See LICENSING.md and EULA.md.